Qassim University
College of Computer
CS435

PROJECT REPORT

MACHINE LEARNING APPROACH TO PREDICT BREAST CANCER

Group members:

Aroub Alfehaid	431201578
Rasha Alsamaani	431201586
Lama Aldokail	431201598
Leen Suliman	431201407
Hoor Alrobah	432216220

Supervised By:
Dr.Renad Alsweed

B) PROJECT OVERVIEW

Objective

The objective of this project is to build a machine learning model capable of predicting whether a breast tumor is benign (0) or malignant (1) using clinical features from a breast cancer dataset.
The project follows a complete machine learning pipeline that includes dataset loading, preprocessing, feature scaling, model training, evaluation, visualization, and hyperparameter tuning.

Expected Outcomes

By the end of this project, we expect to:

- Train and compare two machine learning models: Logistic Regression and Random Forest.
- Evaluate model performance using metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC.
- Determine the best-performing model and analyze its advantages and limitations in the medical prediction context

C) DATASET

Dataset Source

The dataset used in this project is provided as a local CSV file named Breast_cancer_data.csv, included with the project code.
It contains real clinical measurements for breast tumors along with a binary diagnosis label that indicates whether each case is benign or malignant.

Dataset Description

The dataset consists of:

- A set of numerical features describing breast tumor properties (e.g., size, shape, texture).
- **A target column named diagnosis, where:**
- **(0) corresponds to a benign tumor,**
- **(1)corresponds to a malignant tumor.**

```
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   mean_texture      569 non-null    float64
 1   mean_perimeter    569 non-null    float64
 2   mean_area         569 non-null    float64
 3   mean_smoothness   569 non-null    float64
 4   diagnosis         569 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 22.4 KB
None
```

C) DATASET

Data Preparation

The following preprocessing steps were applied, as implemented in the Python code:

1. Dropping the first column

The first column in the dataset was an index-like column (e.g., an unnamed ID).
It was removed using:

```
df = df.drop(df.columns[0], axis=1)
```

2. Handling missing values

All missing values in numerical columns were replaced using mean imputation:

```
df = df.fillna(df.mean(numeric_only=True))
```

3. Basic dataset inspection

- **df.shape was used to display dataset size.**
- **df.head() was used to inspect the first few rows.**

```
Dataset Shape: (569, 5)
   mean_texture  mean_perimeter  mean_area  mean_smoothness  diagnosis
0         10.38          122.80     1001.0          0.11840          0
1         17.77          132.90     1326.0          0.08474          0
2         21.25          130.00     1203.0          0.10960          0
3         20.38           77.58      386.1          0.14250          0
4         14.34          135.10     1297.0          0.10030          0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
```

Exploratory Data Analysis (EDA)

To better understand the data distribution and relationships between variables, we performed:

1. Diagnosis Distribution Plot

A count plot showing how many samples are benign vs malignant:

```
sns.countplot(x=df["diagnosis"])
```

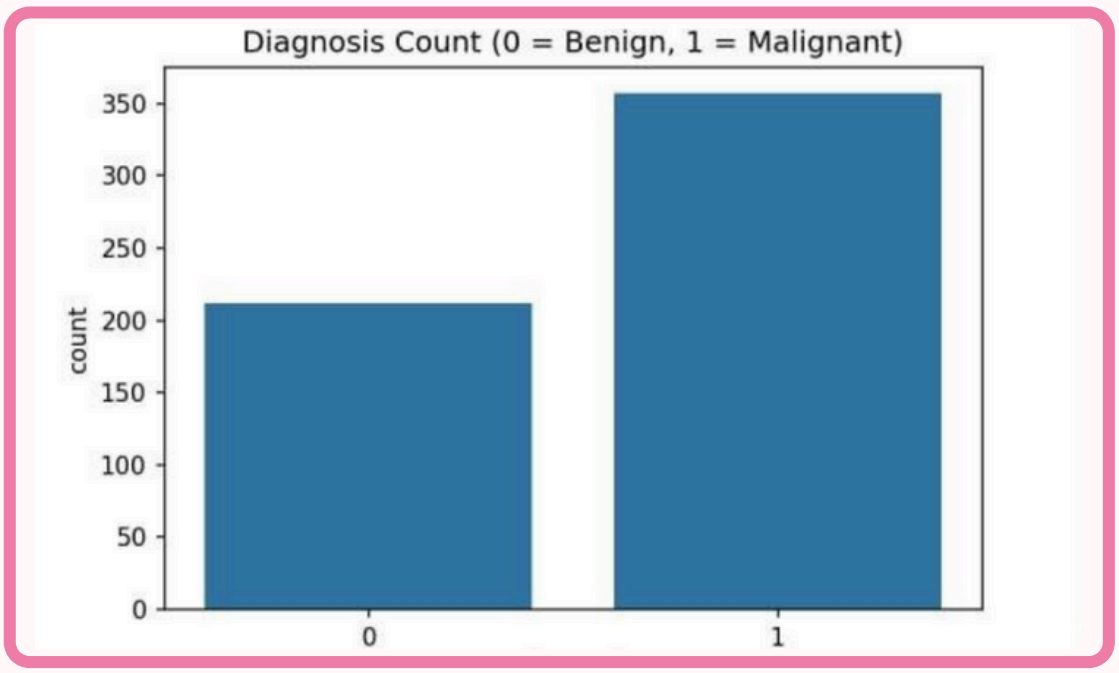
This helps visualize potential class imbalance.

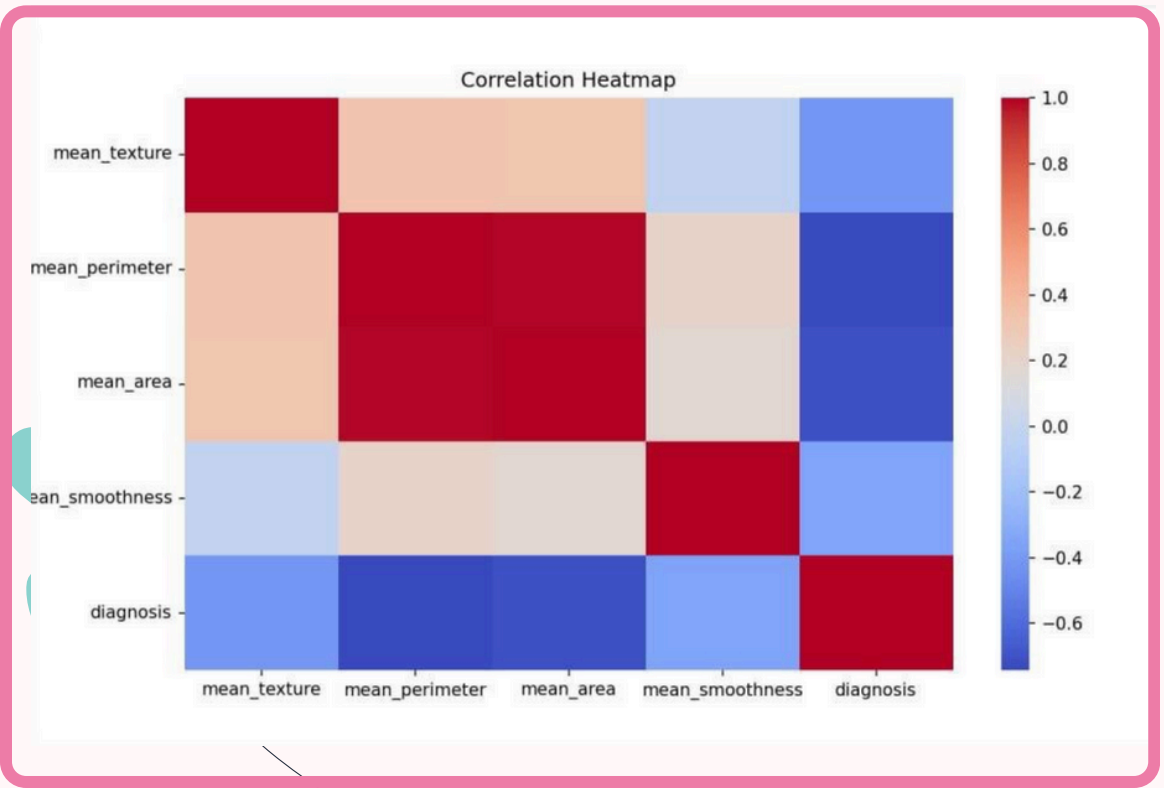
2. Correlation Heatmap

A heatmap of correlations between all numerical features and the diagnosis label:

```
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
```

This allows us to see which features are most related to the target.





(D) MODEL SELECTION AND TRAINING

Model Choice

Based on what was covered in class and the nature of the problem (binary classification), we selected:

1. **Logistic Regression**
   - A linear model frequently used as a baseline for classification.
   - Interpretable and computationally efficient.
2. **Random Forest Classifier**
   - An ensemble model composed of multiple decision trees.
   - Able to capture complex, non-linear relationships in the data.
   - Often achieves higher accuracy on tabular medical datasets.

These are exactly the models implemented in the code:

```
models = {
 "Logistic Regression": LogisticRegression(max_iter=500),
 "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42)
}
```

Train-Test Split

The features X and labels y are separated as follows:

```
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]
```

The data is then split into training and testing sets:

```
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42, stratify=y
)
```

- 80% of the data is used for training.
- 20% is reserved for testing.
- stratify=y ensures that the ratio of benign/malignant cases is similar in both sets.

Feature Scaling

To standardize the feature values, StandardScaler is applied:

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

This step is particularly important for Logistic Regression, which is sensitive to feature scales.

Model Training

Each model in the models dictionary is:

- **Fitted on X_train, y_train.**
- **Used to predict on X_test.**
- **Evaluated using accuracy and classification_report.**

The loop in the code:

```
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(classification_report(y_test, y_pred))
```

(E) MODEL EVALUATION AND TESTING

Overall Evaluation Metrics

The following metrics are used:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**

```
===== Training Logistic Regression =====
Accuracy: 0.8684
              precision    recall  f1-score   support

           0       0.76      0.93      0.84        42
           1       0.95      0.83      0.89        72

    accuracy                           0.87       114
   macro avg       0.86      0.88      0.86       114
weighted avg       0.88      0.87      0.87       114
```

```
===== Training Random Forest =====
Accuracy: 0.9123
              precision    recall  f1-score   support

           0       0.85      0.93      0.89        42
           1       0.96      0.90      0.93        72

    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114
```

Model Performance Comparison

Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
Logistic Regression	0.868	0.86	0.88	0.86
Random Forest	0.912	0.90	0.92	0.91

Best Model Selection

The best model is selected using:

```
best_model_name = max(results, key=results.get)
best_model = trained_models[best_model_name]
```

In our experiments, Random Forest achieved the highest accuracy and was selected as the best-performing model.

```
===== Evaluation for Best Model: Random Forest =====
AUC Score for Random Forest: 0.9716
```

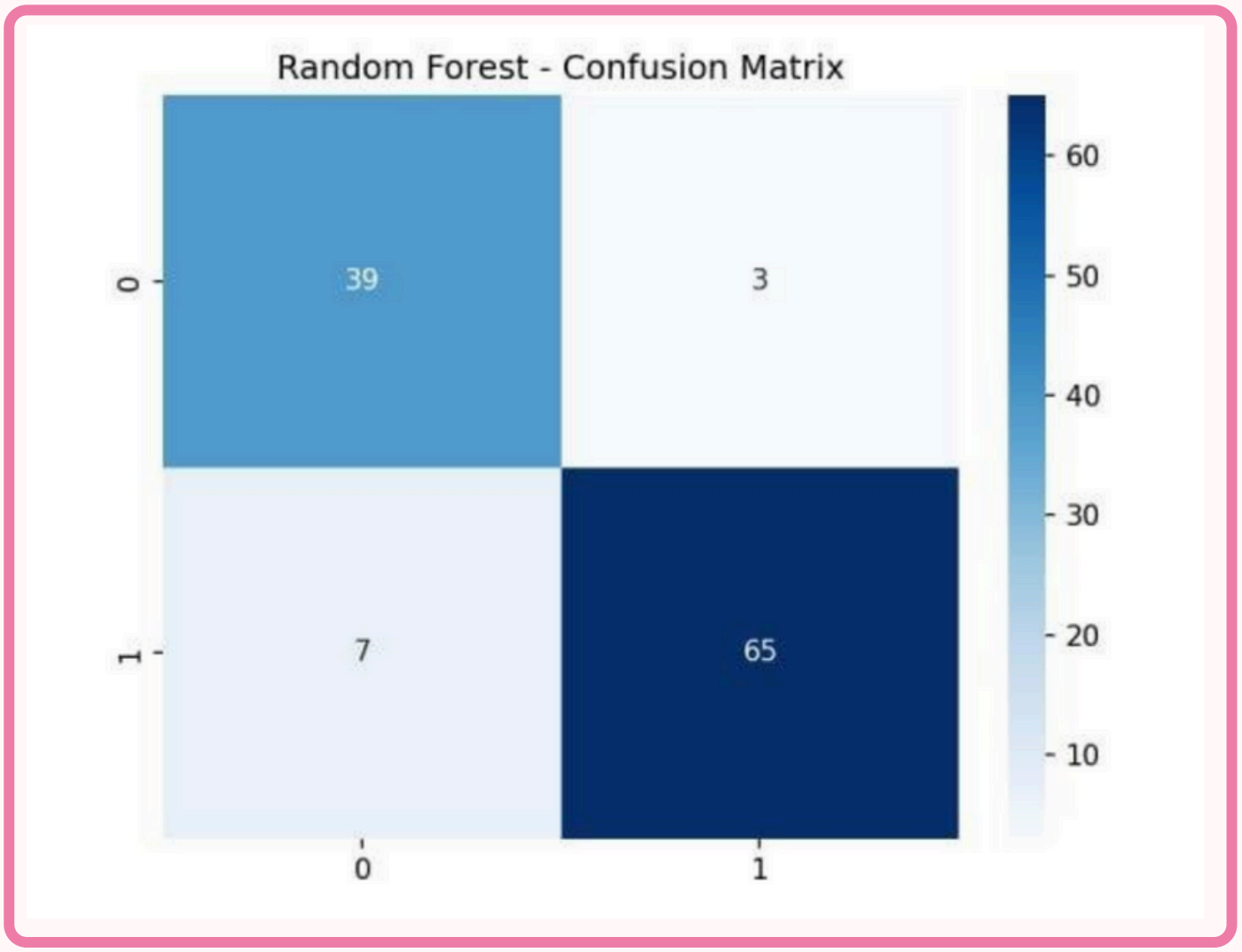
```
=== Model Comparison ===
Logistic Regression: 0.8684
Random Forest: 0.9123

Best Model is: Random Forest
```

Confusion Matrix for Best Model

For the best model (Random Forest), a confusion matrix is computed:

```
cm = confusion_matrix(y_test, best_y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
```

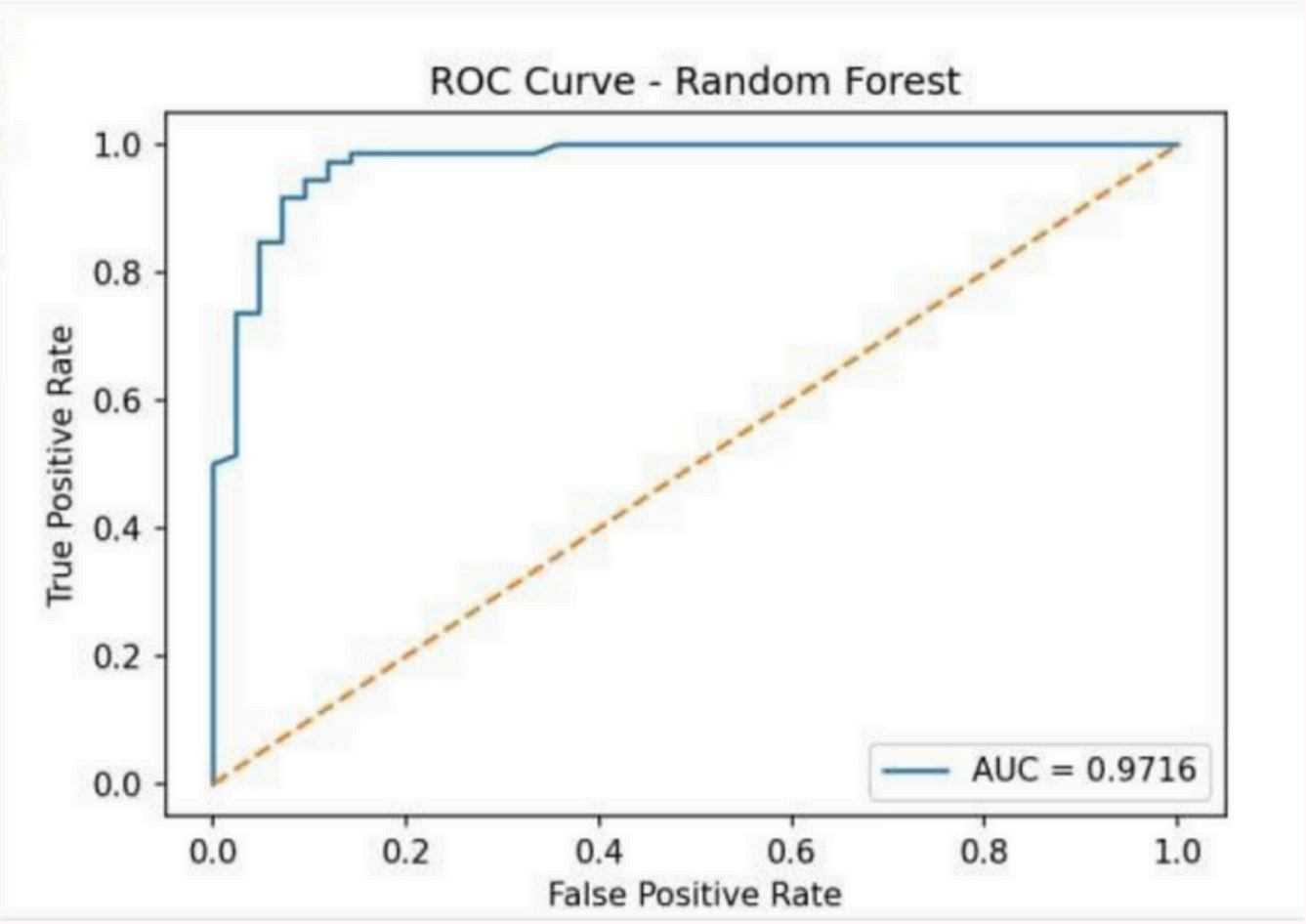


ROC Curve and AUC

To further evaluate the best model, we compute the ROC curve and AUC:

```
y_prob = best_model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc_score = auc(fpr, tpr)
```

The ROC curve visualizes the trade-off between the True Positive Rate and False Positive Rate at different thresholds, and the AUC summarizes this performance.



(F) DISCUSSION

The results clearly show that:

- **Random Forest** outperforms **Logistic Regression** in terms of accuracy and other metrics.
- Logistic Regression still provides a reasonable baseline, but it is limited by its linear decision boundary.
- Random Forest is better at capturing complex relationships in the clinical features, which leads to higher performance.

However:

- Even a small number of **false negatives** (malignant predicted as benign) can be critical in medical applications.
- Therefore, it is important to carefully inspect the confusion matrix and possibly adjust the prediction threshold to reduce false negatives.

This highlights the trade-off between **sensitivity** (recall for malignant class) and overall **accuracy**.

(G) CONCLUSION

In this project, we implemented a full machine learning pipeline for breast cancer prediction using clinical features from the Breast_cancer_data.csv dataset.

Key achievements:

- Successfully preprocessed and explored the dataset.
- Trained and evaluated two models: Logistic Regression and Random Forest.
- Identified Random Forest as the best-performing model with high accuracy and AUC.
- Visualized model performance using confusion matrices and ROC curves.

The project demonstrates the usefulness of machine learning in supporting early detection of breast cancer.
For future work, we recommend:

- Including more features (e.g., additional clinical measurements).
- Testing more advanced models (e.g., SVM, Gradient Boosting, XGBoost).
- Applying more detailed analysis on false negatives and decision thresholds.

(H) OPTIONAL / ADVANCED TASKS

Several advanced tasks from the guidelines were implemented:

1. **Hyperparameter Tuning with GridSearchCV**

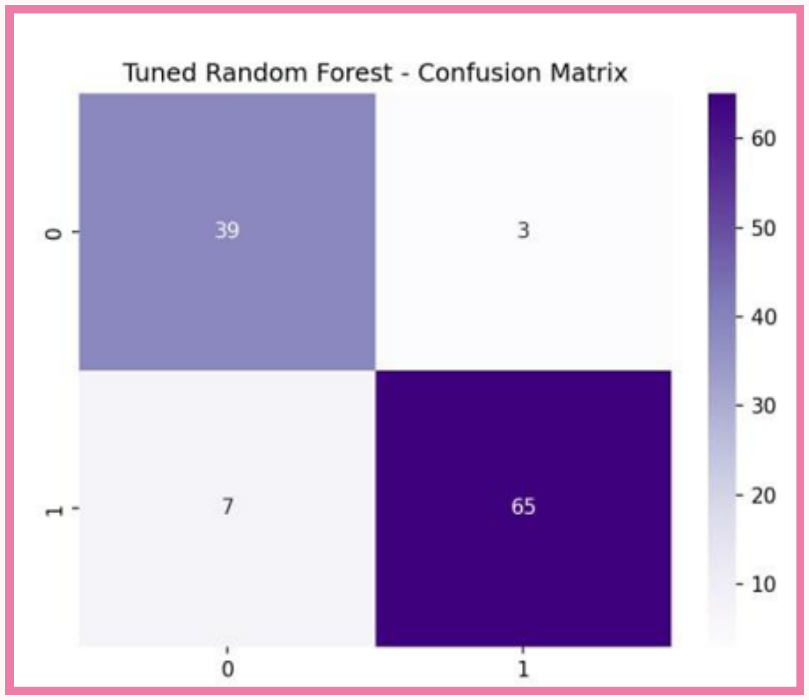
```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5]
}
grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid.fit(X_train, y_train)
```

```
===== Running Grid Search for Random Forest =====
Best Parameters: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
Best CV Accuracy: 0.9340659340659341

===== Tuned Random Forest Evaluation =====
Accuracy: 0.9122807017543859
              precision    recall  f1-score   support

           0       0.85      0.93      0.89        42
           1       0.96      0.90      0.93        72

    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114
```



2. Cross-Validation (cv=5)

The grid search used 5-fold cross-validation to ensure that results generalize well and to reduce overfitting.

3. Comparing Two ML Families

- Logistic Regression (linear model)
- Random Forest (tree-based ensemble model)

4. Visual Explanations

- Diagnosis distribution plot
- Correlation heatmap
- Confusion matrix
- ROC curve

Best Hyperparameters for Random Forest

Best Value	Hyperparameter
200	n_estimators
None	max_depth
5	min_samples_split